



≡ Hybrid and semantic search > Hybrid and semantic search

Hybrid and semantic search

Hybrid and semantic search

📄 Copy page ▼

Combine full-text keyword search with AI-powered semantic search to deliver results that match both exact terms and meaning.

Hybrid search combines two search strategies: full-text search (matching keywords) and semantic search (matching meaning). This gives users the best of both worlds, returning results that are both textually and conceptually relevant.

Embedding models, not LLMs

Semantic search in Meilisearch relies on **embedding models**, not large language models (LLMs). This is an important distinction:

- **Embedding models** convert text into numerical vectors that capture meaning. They are small, fast, and inexpensive to run.
- **LLMs** (like GPT-4 or Claude) generate text and reason about it. They are much larger, slower, and more expensive.

Meilisearch uses embedding models for hybrid and semantic search, making it orders of magnitude cheaper and faster than LLM-based approaches. For conversational AI features that do use LLMs, see conversational search.

How it works

When you configure an embedder, Meilisearch automatically generates vector embeddings for every document in your index. You don't need to compute or manage embeddings



At search time, Meilisearch runs both keyword and semantic search in parallel, then merges the results using a smart scoring system.

Automatic embedding generation

Meilisearch handles the entire embedding pipeline for you:

- **Batching:** documents are grouped and sent to the embedding provider in optimized batches, minimizing API calls and maximizing throughput
- **Caching:** embeddings are stored and only regenerated when document content changes, so re-indexing unchanged documents costs nothing. Note that changing your embedder configuration (switching model, provider, or document template) triggers a full re-embedding of all documents, which may incur significant API costs for large indexes
- **Rate limit handling:** Meilisearch automatically retries when providers return rate limit errors, with no configuration needed
- **Document templates:** you control exactly which fields are embedded using Liquid templates, so the embedding captures the most relevant parts of each document

⚠ Updating embedder settings may trigger a full reindex. When you partially update an index's embedder settings (for example, changing `model`, `source`, `documentTemplate`, `dimensions`, or `pooling`), Meilisearch may reindex all documents and regenerate their embeddings. For large indexes this can take a long time and, with paid providers, incur significant API costs.

`distribution` is a notable exception: changing it does not trigger a reindex.

Smart result ranking

When you perform a hybrid search, Meilisearch does not simply concatenate keyword and semantic results. It uses a scoring system that automatically determines, for each query, whether full-text or semantic results are more relevant:

- A precise query like "iPhone 15 Pro Max 256GB" will naturally favor keyword matches, because the exact terms appear in matching documents
- A descriptive query like "lightweight laptop for travel" will favor semantic matches, because the meaning matters more than the exact words
- Ambiguous queries get a balanced mix of both strategies

You can influence this balance with the [semanticRatio](#) parameter, but the default (0.5) works well for most use cases because Meilisearch's scoring handles the blending intelligently.

When to use hybrid search

Scenario	Best approach
User searches for a product name or SKU	Full-text search
User describes a problem in natural language	Semantic search
Ecommerce product search with varied vocabulary	Hybrid search
Documentation search with technical terms	Hybrid search
FAQ or support knowledge base	Hybrid search

Supported embedder providers

Meilisearch supports a wide range of embedding providers. Some have native integrations, while others are available through the flexible [REST embedder](#) that works with any API.

Native integrations

Provider	Source	Guide
OpenAI	openAi	Configure OpenAI
HuggingFace (local)	huggingFace	Configure HuggingFace

Available via REST embedder

Provider	Guide
Cohere	Configure Cohere
Mistral	Configure Mistral
Google Gemini	Configure Gemini
Cloudflare Workers AI	Configure Cloudflare
Voyage AI	Configure Voyage
AWS Bedrock	Configure Bedrock
HuggingFace Inference Endpoints	Configure HF Inference
Jina	Configure Jina
Any REST API	Configure REST embedder

User-provided embeddings

If you pre-compute embeddings externally (for example, for images or audio content), you can supply them directly. See [search with user-provided embeddings](#).

Embedder field compatibility

Different embedder sources accept different configuration fields. Setting an invalid field for a given source returns an error on settings update.

Field	openAi	huggingFace	ollama	rest
url / apiKey	optional	invalid	optional	required (url)
model	optional	optional	optional	invalid
documentTemplate / documentTemplateMaxBytes	optional	optional	optional	optional

8/1/26, 1:42 PM

Hybrid and semantic search - Meilisearch Documentation

Field	openAi	huggingFace	ollama	rest
 dimensions	optional	optional	optional	optional
pooling	invalid	optional (default useModel)	invalid	invalid
distribution	optional	optional	optional	optional
binaryQuantized	optional	optional	optional	optional

 For composite embedders, these rules apply to each sub-embedder with additional constraints. See [composite embedders](#).

Next steps

Getting started

Configure an embedder and perform your first semantic search

Choose an embedder

Compare providers and pick the right one for your use case

Document templates

Control which document fields are used for embedding generation

Custom hybrid ranking

Tune semanticRatio to balance keyword and semantic results

Was this page helpful?  Yes  No



Product



Meilisearch Cloud

AI search

Pricing

Documentation

Roadmap

Release notes

Comparisons

Trust center

Company

Blog

Customers

Careers

Code of Conduct

Privacy Policy

Terms of Use

Swag

Contact us

GitHub

Discord

Newsletter

Helpdesk

Powered by **mintlify**

